

# Employee Management JDBC Project:

## 1. Introduction

The Employee Management System is a console-based Java application developed to manage employee information efficiently. It allows users to store, retrieve, update, and delete employee records from a MySQL database using JDBC connectivity. The Employee Management System is a console-based Java application designed to manage employee records in a company database. It uses JDBC to connect Java with a MySQL database and follows a layered architecture to keep the project organized and easy to understand.

## 2. Objective

- To perform CRUD (Create, Read, Update, Delete) operations on employee records
- To demonstrate database connectivity using JDBC
- To implement layered architecture (DTO, DAO, Service, Main)

## 3. Technology Used

- Programming Language: Core Java
- Database: MySQL
- Connectivity: JDBC (Java Database Connectivity)
- IDE: Eclipse / IntelliJ

## 4. Database Design

### 4.1 Create Database

```
CREATE DATABASE company;  
USE company;
```

### 4.2 Create Employee Table

```
CREATE TABLE employee (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100) NOT NULL,  
    dob DATE,  
    gender VARCHAR(10),  
    department VARCHAR(50),  
    salary DOUBLE  
);
```

### 4.3 Sample Records

```
INSERT INTO employee (name, dob, gender, department, salary) VALUES  
('Amit Sharma', '1995-04-12', 'Male', 'IT', 55000),
```

```
( 'Priya Nair', '1998-08-22', 'Female', 'HR', 48000),
( 'Rahul Verma', '1993-01-15', 'Male', 'Finance', 62000),
( 'Sneha Reddy', '1997-03-30', 'Female', 'Marketing', 50000),
( 'Karan Mehta', '1992-11-05', 'Male', 'IT', 70000),
( 'Anjali Gupta', '1999-07-18', 'Female', 'HR', 46000),
( 'Rohit Kumar', '1994-02-09', 'Male', 'Finance', 64000),
( 'Neha Singh', '1996-06-25', 'Female', 'Marketing', 52000),
( 'Vikram Joshi', '1991-09-14', 'Male', 'IT', 75000),
( 'Pooja Iyer', '1998-12-02', 'Female', 'HR', 47000);
```

| Column Name       | Data Type                         | Description        |
|-------------------|-----------------------------------|--------------------|
| <b>id</b>         | INT (Primary Key, Auto Increment) | Unique Employee ID |
| <b>name</b>       | VARCHAR(100)                      | Employee Full Name |
| <b>dob</b>        | DATE                              | Date of Birth      |
| <b>gender</b>     | VARCHAR(10)                       | Gender of Employee |
| <b>department</b> | VARCHAR(50)                       | Department Name    |
| <b>salary</b>     | DOUBLE                            | Employee Salary    |

## 5. Layered Architecture Explanation

### DTO (Data Transfer Object)

DTO classes are simple Java classes that hold data and transfer it between layers without business logic. In this project, `Employee.java` is the DTO.

### DAO (Data Access Object)

DAO defines database operations such as insert, update, delete, and fetch. It separates database logic from business logic.

### DAO Implementation (DAOImpl)

This is the actual class that implements DAO methods using JDBC queries. In this project, `EmployeeDAO.java` acts as the DAO implementation.

### Service Layer

The Service layer contains business logic and user interaction. It receives input from the user and calls DAO methods. Here, `EmployeeService.java` is the service layer.

## Main Layer

The Main class provides the menu-driven interface for users to interact with the system. EmployeeApp.java is the entry point of the application.

## 6. Source Code

### 6.1 Employee.java (DTO)

```
package project_anp;
import java.sql.Date;

public class Employee {
    private int id;
    private String name;
    private Date dob;
    private String gender;
    private String department;
    private double salary;

    public Employee(int id, String name, Date dob, String gender, String
department, double salary) {
        this.id = id;
        this.name = name;
        this.dob = dob;
        this.gender = gender;
        this.department = department;
        this.salary = salary;
    }

    public Employee(String name, Date dob, String gender, String department,
double salary) {
        this.name = name;
        this.dob = dob;
        this.gender = gender;
        this.department = department;
        this.salary = salary;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public Date getDob() { return dob; }
    public String getGender() { return gender; }
    public String getDepartment() { return department; }
    public double getSalary() { return salary; }
}
```

## 6.2 DBConnection.java

```
package project_anp;
import java.sql.Connection;
import java.sql.DriverManager;

public class DBConnection {

    public static void main(String[]args) throws ClassNotFoundException{

        Class.forName("com.mysql.cj.jdbc.Driver");

    }

    private static final String URL = "jdbc:mysql://localhost:3306/company";
    private static final String USER = "root";
    private static final String PASSWORD = "kavya29-";

    public static Connection getConnection() throws Exception {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

## 6.3 EmployeeDAO.java

```
package project_anp;
import java.sql.*;
import java.util.*;

public class EmployeeDAO {

    public void addEmployee(Employee e) {
        String sql = "INSERT INTO employee(name, dob, gender, department, salary) VALUES (?, ?, ?, ?, ?)";
        try (Connection con = DBConnection.getConnection(); PreparedStatement
ps = con.prepareStatement(sql)) {
            ps.setString(1, e.getName());
            ps.setDate(2, e.getDob());
            ps.setString(3, e.getGender());
            ps.setString(4, e.getDepartment());
            ps.setDouble(5, e.getSalary());
            ps.executeUpdate();
        } catch (Exception ex) { ex.printStackTrace(); }
    }

    public List<Employee> getAllEmployees() {
        List<Employee> list = new ArrayList<>();
        String sql = "SELECT * FROM employee";
        try (Connection con = DBConnection.getConnection(); Statement st =
```

```

con.createStatement(); ResultSet rs = st.executeQuery(sql)) {
    while (rs.next()) {
        list.add(new Employee(rs.getInt("id"), rs.getString("name"),
rs.getDate("dob"), rs.getString("gender"), rs.getString("department"),
rs.getDouble("salary")));
    }
    } catch (Exception ex) { ex.printStackTrace(); }
    return list;
}

public boolean deleteEmployee(int id) {
    String sql = "DELETE FROM employee WHERE id=?";
    try (Connection con = DBConnection.getConnection(); PreparedStatement
ps = con.prepareStatement(sql)) {
        ps.setInt(1, id);
        return ps.executeUpdate() > 0;
    } catch (Exception ex) { ex.printStackTrace(); }
    return false;
}

public boolean updateSalary(int id, double salary) {
    String sql = "UPDATE employee SET salary=? WHERE id=?";
    try (Connection con = DBConnection.getConnection(); PreparedStatement
ps = con.prepareStatement(sql)) {
        ps.setDouble(1, salary);
        ps.setInt(2, id);
        return ps.executeUpdate() > 0;
    } catch (Exception ex) { ex.printStackTrace(); }
    return false;
}

public Employee getEmployeeById(int id) {
    String sql = "SELECT * FROM employee WHERE id=?";
    try (Connection con = DBConnection.getConnection(); PreparedStatement
ps = con.prepareStatement(sql)) {
        ps.setInt(1, id);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) return new Employee(rs.getInt("id"),
rs.getString("name"), rs.getDate("dob"), rs.getString("gender"),
rs.getString("department"), rs.getDouble("salary"));
    } catch (Exception ex) { ex.printStackTrace(); }
    return null;
}
}

```

## 6.4 EmployeeService.java

```

package project_anp;
import java.sql.Date;
import java.util.*;

```

```

public class EmployeeService {
    private final EmployeeDAO dao = new EmployeeDAO();
    private final Scanner sc = new Scanner(System.in);

    public void addEmployee() {
        System.out.print("Name: "); String name = sc.nextLine();
        System.out.print("DOB (yyyy-mm-dd): "); Date dob =
Date.valueOf(sc.nextLine());
        System.out.print("Gender: "); String gender = sc.nextLine();
        System.out.print("Department: "); String dept = sc.nextLine();
        System.out.print("Salary: "); double salary = sc.nextDouble();
sc.nextLine();
        dao.addEmployee(new Employee(name, dob, gender, dept, salary));
    }

    public void viewEmployees() {
        dao.getAllEmployees().forEach(e -> System.out.println(e.getId()+" |
"+e.getName()+" | "+e.getDepartment()+" | "+e.getSalary()));
    }

    public void deleteEmployee() {
        System.out.print("Enter ID to delete: "); int id = sc.nextInt();
sc.nextLine();
        System.out.println(dao.deleteEmployee(id) ? "Deleted!" : "Not
found!");
    }

    public void updateEmployeeSalary() {
        System.out.print("Employee ID: "); int id = sc.nextInt();
        System.out.print("New Salary: "); double salary = sc.nextDouble();
sc.nextLine();
        System.out.println(dao.updateSalary(id, salary) ? "Salary updated!" :
"Employee not found!");
    }

    public void searchEmployee() {
        System.out.print("Employee ID: "); int id = sc.nextInt();
sc.nextLine();
        Employee e = dao.getEmployeeById(id);
        if (e != null) System.out.println(e.getId()+" | "+e.getName()+" |
"+e.getDepartment()+" | "+e.getSalary());
        else System.out.println("Employee not found!");
    }
}

```

## 6.5 EmployeeApp.java

```

package project_anp;
import java.util.*;

```

```

public class EmployeeApp {
    public static void main(String[] args) {
        EmployeeService service = new EmployeeService();
        Scanner sc = new Scanner(System.in);
        while (true) {
            System.out.println("\n--- EMPLOYEE MENU ---");
            System.out.println("1. Add Employee");
            System.out.println("2. View Employees");
            System.out.println("3. Delete Employee");
            System.out.println("4. Update Employee Salary");
            System.out.println("5. Search Employee by ID");
            System.out.println("6. Exit");
            System.out.print("Enter choice: ");
            int ch = sc.nextInt(); sc.nextLine();
            switch (ch) {
                case 1 -> service.addEmployee();
                case 2 -> service.viewEmployees();
                case 3 -> service.deleteEmployee();
                case 4 -> service.updateEmployeeSalary();
                case 5 -> service.searchEmployee();
                case 6 -> { System.out.println("Goodbye!"); return; }
                default -> System.out.println("Invalid choice!");
            }
        }
    }
}

```

## 7. Technology Used

- To perform CRUD (Create, Read, Update, Delete) operations on employee records
- To demonstrate database connectivity using JDBC
- To implement layered architecture (DTO, DAO, Service, Main)

## 8. Features of the System

### 1. Add Employee

This feature allows the user to enter new employee details such as **name, date of birth, gender, department, and salary**.

The data is then stored in the **MySQL database** using an `INSERT` query.

## 2. View Employees

This feature retrieves and displays **all employee records** from the database. It uses a `SELECT` query and shows the employee list in the console.

## 3. Delete Employee

This feature allows the user to remove an employee record by entering the **Employee ID**. The record is deleted using a `DELETE` query.

## 4. Update Employee Salary

This feature allows updating the **salary** of an existing employee by entering their **Employee ID** and new salary amount. It uses an `UPDATE` query to modify the database record.

## 5. Search Employee by ID

This feature helps the user find a specific employee using their **Employee ID**. The system fetches the matching record using a `SELECT WHERE` query and displays the employee details.

## 6. Exit

This option safely closes the application and stops the program execution.

## 9. Sample Output

Below is a sample console output of the application:

```
--- EMPLOYEE MENU ---
1. Add Employee
2. View Employees
3. Delete Employee
4. Update Employee Salary
5. Search Employee by ID
6. Exit
Enter choice: 2

1 | Amit Sharma | IT | 55000.0
2 | Priya Nair | HR | 48000.0
3 | Rahul Verma | Finance | 62000.0

--- EMPLOYEE MENU ---
Enter choice: 5
Enter Employee ID: 1
```



Employee Found:  
1 | Amit Sharma | IT | 55000.0

--- EMPLOYEE MENU ---  
Enter choice: 4  
Employee ID: 1  
New Salary: 60000  
Salary updated successfully!

--- EMPLOYEE MENU ---  
Enter choice: 3  
Enter ID to delete: 2  
Deleted!

--- EMPLOYEE MENU ---  
Enter choice: 6  
Goodbye!

## 10. Conclusion

This project demonstrates how Java interacts with MySQL using JDBC and layered architecture. This project demonstrates how Java interacts with MySQL using JDBC and layered architecture.